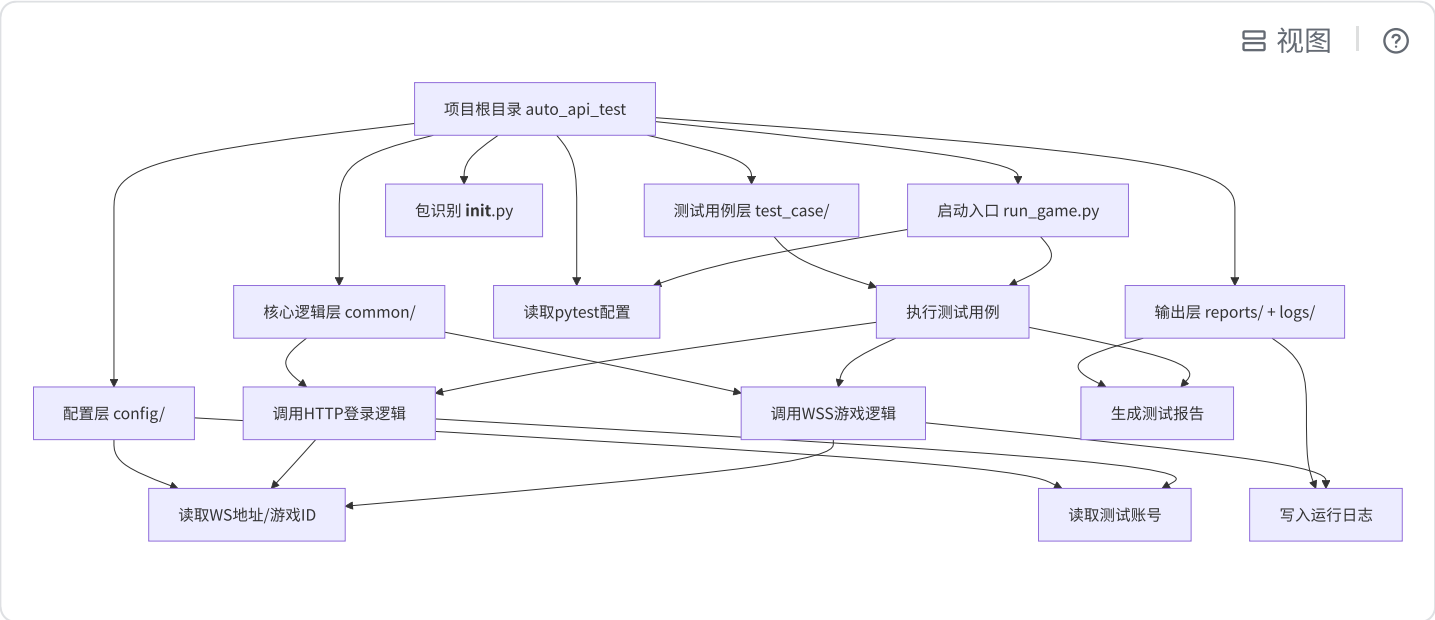
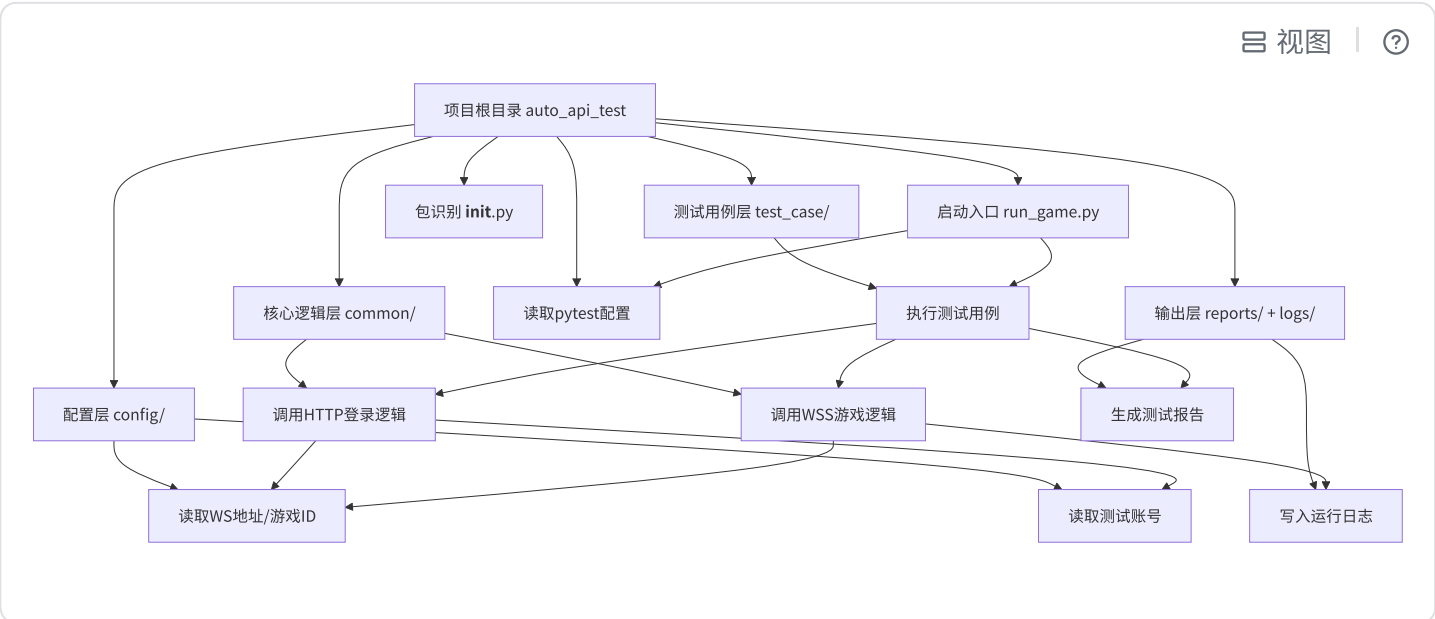
游戏自动化测试项目 - 架构图与流程图

本文档包含项目整体架构图、核心执行流程图，清晰展示各模块关联、执行逻辑，适配团队交付、新人快速理解项目运行机制。

一、项目整体架构图

说明：架构图展示各文件夹、核心文件的关联关系，明确数据流向和依赖关系。

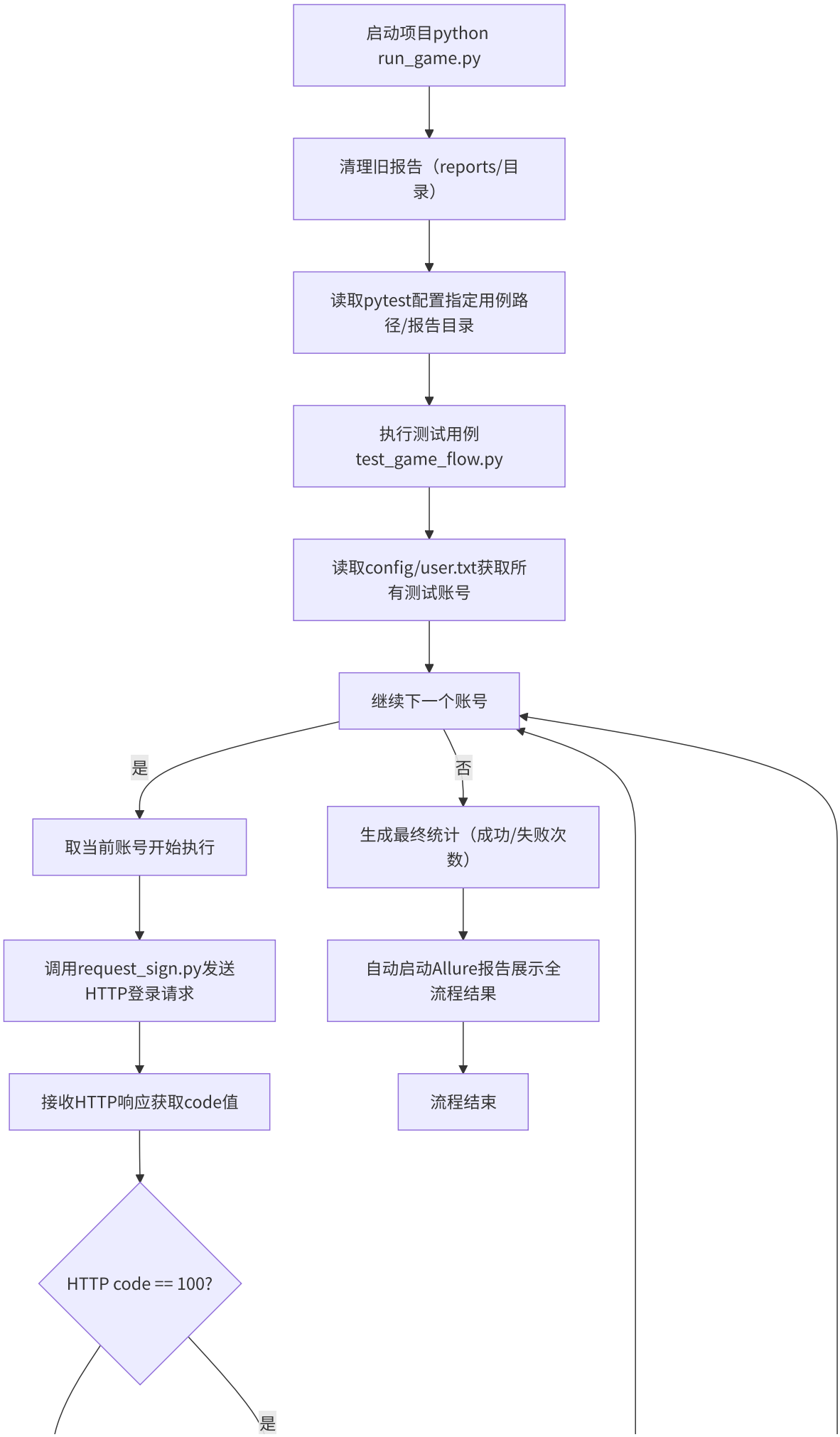


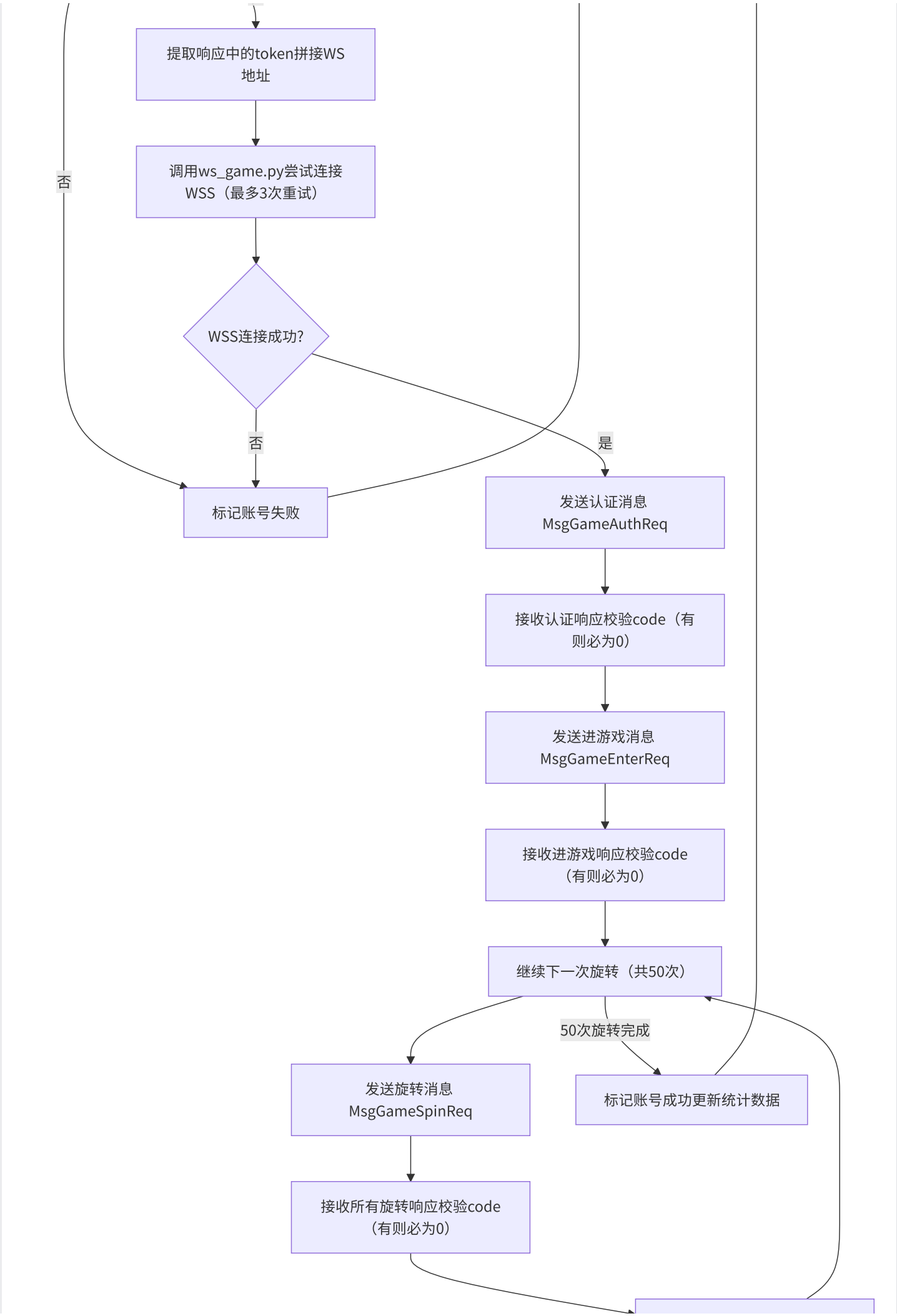


二、核心执行流程图

说明：流程图展示从启动项目到执行完成的全流程，包含各步骤逻辑、校验规则、重试机制。

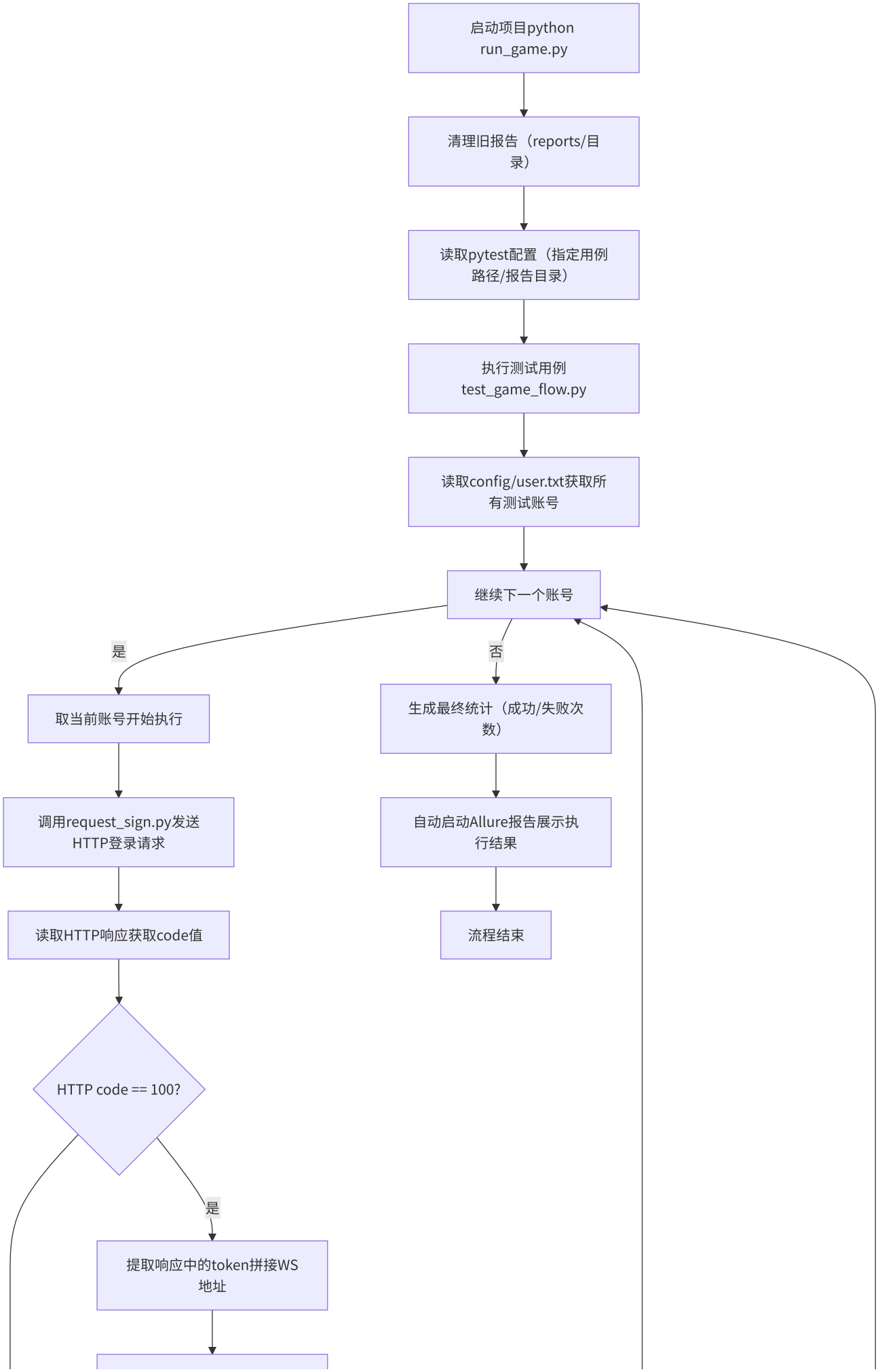
视图

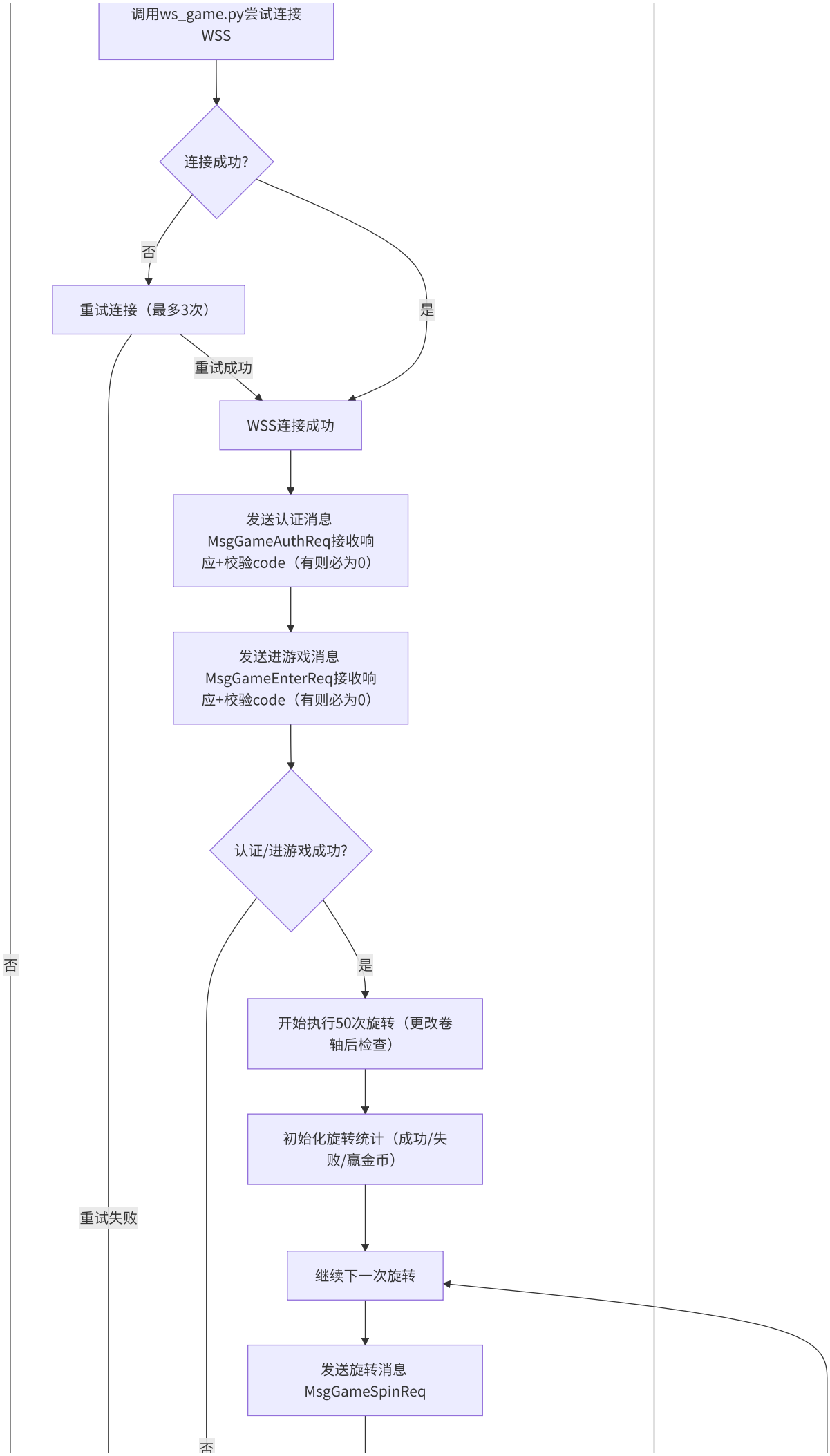

启动项目python run_game.py
清理旧报告（reports/目录）
读取pytest配置指定用例路径/报告目录
执行测试用例 test_game_flow.py
读取config/user.txt获取所有测试账号
继续下一个账号
是
否
取当前账号开始执行
生成最终统计（成功/失败次数）
调用request_sign.py发送HTTP登录请求
自动启动Allure报告展示全流程结果
接收HTTP响应获取code值
流程结束
HTTP code == 100?
是

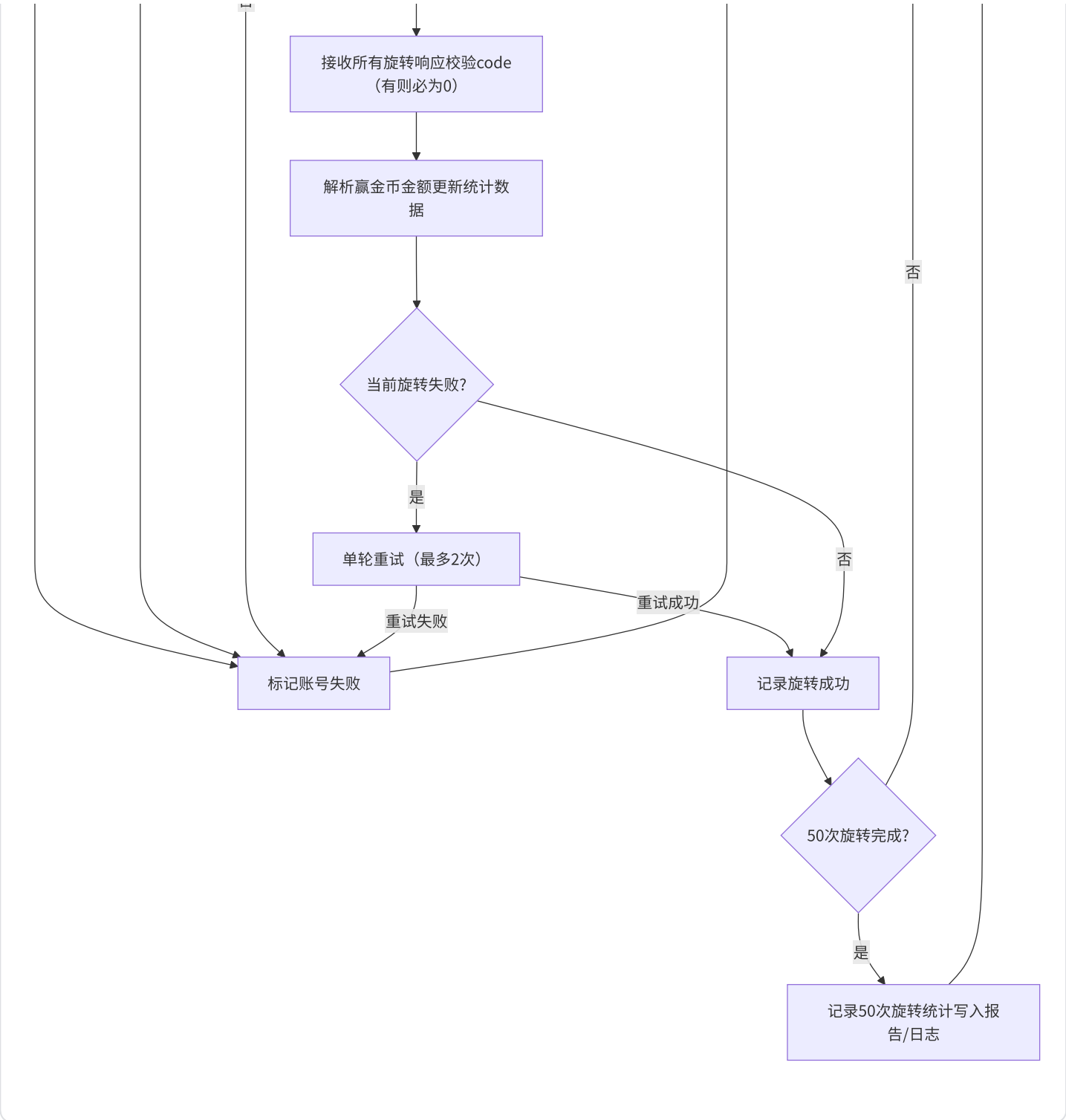

提取响应中的token拼接WS地址
调用ws_game.py尝试连接WSS（最多3次重试）
WSS连接成功?
否
是
否
标记账号失败
发送认证消息MsgGameAuthReq
接收认证响应校验code（有则必为0）
发送进游戏消息MsgGameEnterReq
接收进游戏响应校验code（有则必为0）
继续下一次旋转（共50次）
50次旋转完成
发送旋转消息MsgGameSpinReq
标记账号成功更新统计数据
接收所有旋转响应校验code（有则必为0）

记录旋转结果/赢金币统计写入日志/报告

视图


启动项目python run_game.py
清理旧报告（reports/目录）
读取pytest配置（指定用例路径/报告目录）
执行测试用例 test_game_flow.py
读取config/user.txt获取所有测试账号
继续下一个账号
是
否
取当前账号开始执行
调用request_sign.py发送HTTP登录请求
读取HTTP响应获取code值
HTTP code == 100?
是
提取响应中的token拼接WS地址
生成最终统计（成功/失败次数）
自动启动Allure报告展示执行结果
流程结束


调用ws_game.py尝试连接WSS
连接成功?
否
是
重试连接（最多3次）
重试成功
WSS连接成功
发送认证消息MsgGameAuthReq接收响应+校验code（有则必为0）
发送进游戏消息MsgGameEnterReq接收响应+校验code（有则必为0）
认证/进游戏成功?
否
是
开始执行50次旋转（更改卷轴后检查）
初始化旋转统计（成功/失败/赢金币）
重试失败
继续下一次旋转
发送旋转消息MsgGameSpinReq
否



三、图表说明

1. 架构图核心说明

- 配置层（config/）：所有可配置项集中管理，无需修改代码，降低维护成本。
- 核心逻辑层（common/）：封装HTTP、WSS核心逻辑，供测试用例调用，实现代码复用。
- 测试用例层（test_case/）：仅负责流程串联，不包含具体业务逻辑，便于维护。
- 输出层（reports/、logs/）：自动生成报告和日志，用于结果查看、问题排查。

2. 流程图核心说明

- 全流程自动化：从启动到报告生成，无需人工干预，一键完成。
- 校验规则：严格遵循HTTP code=100、WS有code必为0的规则，异常自动拦截。
- 重试机制：WSS连接重试3次、单轮旋转重试2次，提升脚本稳定性。
- 统计功能：自动统计50次旋转的成功/失败次数、总赢金币，直观展示测试结果。

四、使用说明

1. 本文档可直接放在项目根目录，与「项目说明文档.md」「快速上手使用步骤.md」配套使用。
2. Mermaid语法支持：VS Code、Typora、GitLab等工具可直接渲染图表，无需额外安装插件。
3. 适配场景：团队交付、新人培训、项目复盘，清晰展示项目架构和执行逻辑。

（注：文档部分内容可能由 AI 生成）